

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN

PROYECTO FIN DE CURSO (PFC) — ENTREGA 2

PROYECTO:

**TIENDATECH: PROPUESTA Y ANÁLISIS DEL
SISTEMA DISTRIBUIDO**

MATERIA:

APLICACIONES DISTRIBUIDAS

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

CURSO / PERÍODO ACADÉMICO:

7mo A — PERÍODO ACADÉMICO 2026

INTEGRANTES Y ROLES:

Arquitecto: Jhinson Stalyn Aucatoma Celorio
Líder de Desarrollo: Jeremy Ruperto Gaibor Rodriguez
Responsable de Calidad: Andy Paul Sanchez Pilaloa
Responsable de Documentación: José Alejandro Lozano Morales

FECHA DE ENTREGA:

6 DE JUNIO 2026

Resumen

El proyecto **TiendaTech** propone el diseño e implementación de una plataforma de comercio electrónico especializada en la venta de componentes de hardware y el ensamblado personalizado de computadoras en Ecuador. La problemática abordada se relaciona con la dificultad que enfrentan los usuarios al seleccionar componentes compatibles, situación que puede generar errores de compra, devoluciones y pérdidas económicas. Como solución, se plantea una arquitectura distribuida basada en microservicios que permite gestionar de forma independiente los procesos de autenticación, catálogo de productos, carrito de compras, inventario, pedidos, pagos y notificaciones. Además, el sistema incorpora un motor de compatibilidad de hardware y un asistente inteligente para apoyar al usuario durante el proceso de selección de componentes. En esta segunda entrega se presenta el diseño arquitectónico del sistema mediante diagramas C4, la definición inicial de servicios REST, la especificación del contrato OpenAPI 3.0 y la estrategia de contenerización utilizando Docker. La propuesta busca garantizar escalabilidad, mantenibilidad y alta disponibilidad, estableciendo las bases necesarias para la implementación experimental del sistema en las siguientes etapas del proyecto.

Palabras clave: Microservicios, Comercio Electrónico, Sistemas Distribuidos, API REST, Docker, Compatibilidad de Hardware.

Abstract

TiendaTech proposes the design and implementation of a national e-commerce platform specialized in the sale of computer hardware components and custom PC assembly services in Ecuador. The problem addressed is the difficulty users face when selecting compatible hardware components, which may lead to purchasing errors, product returns, and financial losses. To address this issue, a distributed architecture based on microservices is proposed, enabling the independent management of authentication, product catalog, shopping cart, inventory, orders, payments, and notification processes. Additionally, the system incorporates a hardware compatibility engine and an intelligent assistant to support users during the component selection process. This second project deliverable presents the architectural design of the system through C4 diagrams, the definition of REST services, the initial OpenAPI 3.0 contract specification, and the containerization strategy using Docker. The proposed solution aims to ensure scalability, maintainability, and high availability while establishing the foundation for the experimental implementation phases of the project.

Keywords: Microservices, Electronic Commerce, Distributed Systems, REST API, Docker, Hardware Compatibility.

1. Registro de Cambios respecto a la Entrega 1

Con base en la retroalimentación de la Entrega 1, se realizaron mejoras en el diseño arquitectónico y la documentación técnica de **TiendaTech**, con el objetivo de incrementar el nivel de detalle y formalizar las decisiones de diseño. La Tabla 1 resume los cambios más relevantes.

Cuadro 1: Registro de cambios respecto a la Entrega 1

Aspecto	Entrega 1	Entrega 2	Justificación
Arquitectura	Visión general distribuida	Microservicios formalmente definidos	Permite escalar módulos de forma independiente y reduce el acoplamiento.
Modelado C4	Niveles 1 y 2	Incorporación del Nivel 3	Describe con mayor detalle los componentes internos y sus responsabilidades.
Comunicación entre servicios	Descripción conceptual	Servicios REST formalmente definidos	Establece mecanismos claros de integración entre microservicios.
Documentación de APIs	No definida	Contratos OpenAPI 3.0 por microservicio	Estandariza la documentación y reduce ambigüedades en la implementación.
Despliegue	Descripción conceptual	Docker y Docker Compose	Garantiza portabilidad y despliegue consistente del sistema distribuido.
Decisiones arquitectónicas	Justificación general	Registros ADR formalizados	Documenta las decisiones técnicas adoptadas y sus consecuencias.
Calidad del sistema	No definida formalmente	Métricas ISO/IEC 25010	Establece criterios objetivos de rendimiento, seguridad y mantenibilidad.

Estos ajustes permiten que la propuesta evolucione desde una visión conceptual hacia una arquitectura distribuida formalmente documentada, lista para las etapas de implementación y validación del Proyecto Fin de Curso.

2. Introducción

2.1. Contexto y Problema Cuantificado

En la última década, el comercio electrónico en el Ecuador ha experimentado un crecimiento exponencial. Según datos oficiales de la Cámara Ecuatoriana de Comercio Electrónico (CE-CE), las transacciones digitales en el país han alcanzado una facturación anual que supera los 4,000 millones de dólares, donde la categoría de bienes tecnológicos y hardware informático se posiciona de forma consistente entre los tres sectores con mayor volumen de transacciones y demanda de mercado. Dentro de este ecosistema macroeconómico digital, el aprovisionamiento de computación personalizada de alto rendimiento exige plataformas comerciales que superen las capacidades de los catálogos estáticos tradicionales.

El despliegue de plataformas nacionales complejas orientadas al comercio especializado requiere de la evolución de las arquitecturas de software tradicionales hacia sistemas distribuidos capaces de procesar información heterogénea en tiempo real [1]. En este escenario nace **TiendaTech**, una plataforma centralizada de cobertura nacional diseñada no solo para capturar una cuota del flujo comercial cuantificado del hardware en el país, sino para automatizar la asistencia técnica mediante sistemas de software interactivos.

2.2. Definición del Problema y Brecha de Requerimientos Formales

El proceso de selección e integración de componentes para el armado de un computador representa un desafío crítico debido a la alta asimetría de información y a las estrictas dependencias de compatibilidad física, eléctrica y lógica entre las piezas. Un usuario se enfrenta al riesgo constante de adquirir componentes que presentan colisiones tecnológicas insalvables (como la disparidad de zócalos entre procesador y placa madre). En el contexto del e-commerce ecuatoriano actual, la mayoría de los sistemas operan bajo esquemas monolíticos que aíslan el inventario de las especificaciones de ingeniería del producto.

Aquí se identifica una evidente **brecha de requerimientos formales (Brecha RQ)**: las plataformas comerciales vigentes cumplen con requerimientos transaccionales básicos (gestión de carrito, pasarela de pagos), pero carecen de requerimientos lógicos y formales de validación cruzada en tiempo real. Al no existir especificaciones formales automatizadas que validen las restricciones tecnológicas de los productos antes de la compra, el error del consumidor se detecta únicamente tras la entrega física, lo que desencadena ciclos ineficientes de devoluciones y pérdidas económicas para la empresa, evidenciando un vacío técnico que los sistemas monolíticos convencionales no pueden resolver.

2.3. Justificación de la Solución y Contribución Delimitada

Para mitigar la brecha identificada, **TiendaTech** se concibe bajo el paradigma de un sistema distribuido fundamentado en una arquitectura de microservicios elásticos. La naturaleza síncrona de las validaciones de hardware, el cálculo de impuestos y la facturación electrónica nacional exigen un aislamiento de procesos para garantizar alta disponibilidad y tolerancia a fallos [2]. El motor de compatibilidad se separa en un microservicio especializado que soluciona los conflictos de hardware mediante modelos lógicos de programación por restricciones [3], interpretando las reglas ante el usuario mediante enfoques de co-clustering explicable [4]. Asimismo, la brecha de soporte se cubre con un microservicio de Asistente Inteligente basado en Modelos de Lenguaje Grandes (LLMs) y arquitecturas RAG [5].

La **contribución delimitada** del presente proyecto durante este ciclo académico se restringe formalmente al diseño arquitectónico, modelado y despliegue del ecosistema distribuido centralizado para TiendaTech. Esto comprende la especificación del API Gateway, la construcción de las bases de datos distribuidas y la implementación del núcleo lógico de los módulos de autenticación, carrito de compras, motor de compatibilidad y la interfaz cognitiva base del asistente. Con esta delimitación, el proyecto no pretende resolver la cadena de suministro física, sino entregar una solución de software de alta disponibilidad que resuelva de raíz la brecha de validación lógica en el comercio electrónico tecnológico del país.

3. Estado del Arte

El desarrollo de plataformas modernas de comercio electrónico optimizadas para la configuración de productos complejos ha impulsado una notable evolución en la literatura científica. A continuación, se presenta una síntesis integradora de los enfoques metodológicos actuales, estructurada mediante el mapeo de problemas y tecnologías de vanguardia indexadas entre los años 2020 y 2026.

4. Modelado Arquitectónico mediante Diagramas C4

Con el objetivo de representar la estructura general del ecosistema distribuido de **TiendaTech**, se emplea el modelo arquitectónico C4, el cual permite describir sistemas de software complejos mediante diferentes niveles de abstracción.

4.1. Diagrama C4-L1: Contexto del Sistema

El diagrama C4 de nivel 1 permite visualizar la interacción global entre los usuarios externos y la plataforma distribuida **TiendaTech**. En este nivel se identifican los actores principales, así como los servicios externos en el flujo general de operación del sistema.

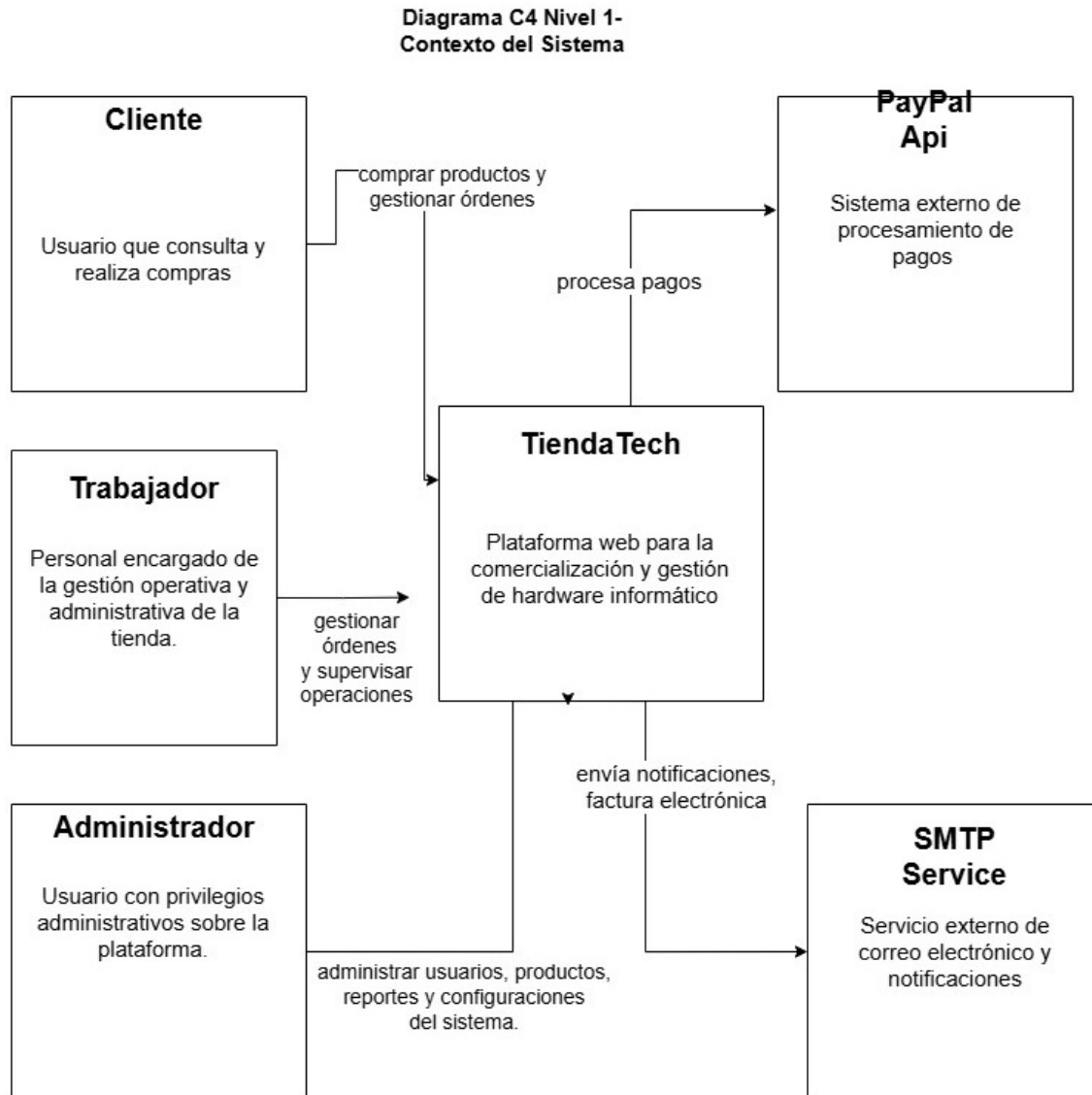


Figura 1: Diagrama C4-L1 del sistema distribuido TiendaTech.

4.2. Diagrama C4-L2: Diagrama de Contenedores

El diagrama C4 de nivel 2 representa la descomposición interna de la arquitectura distribuida de **TiendaTech**.

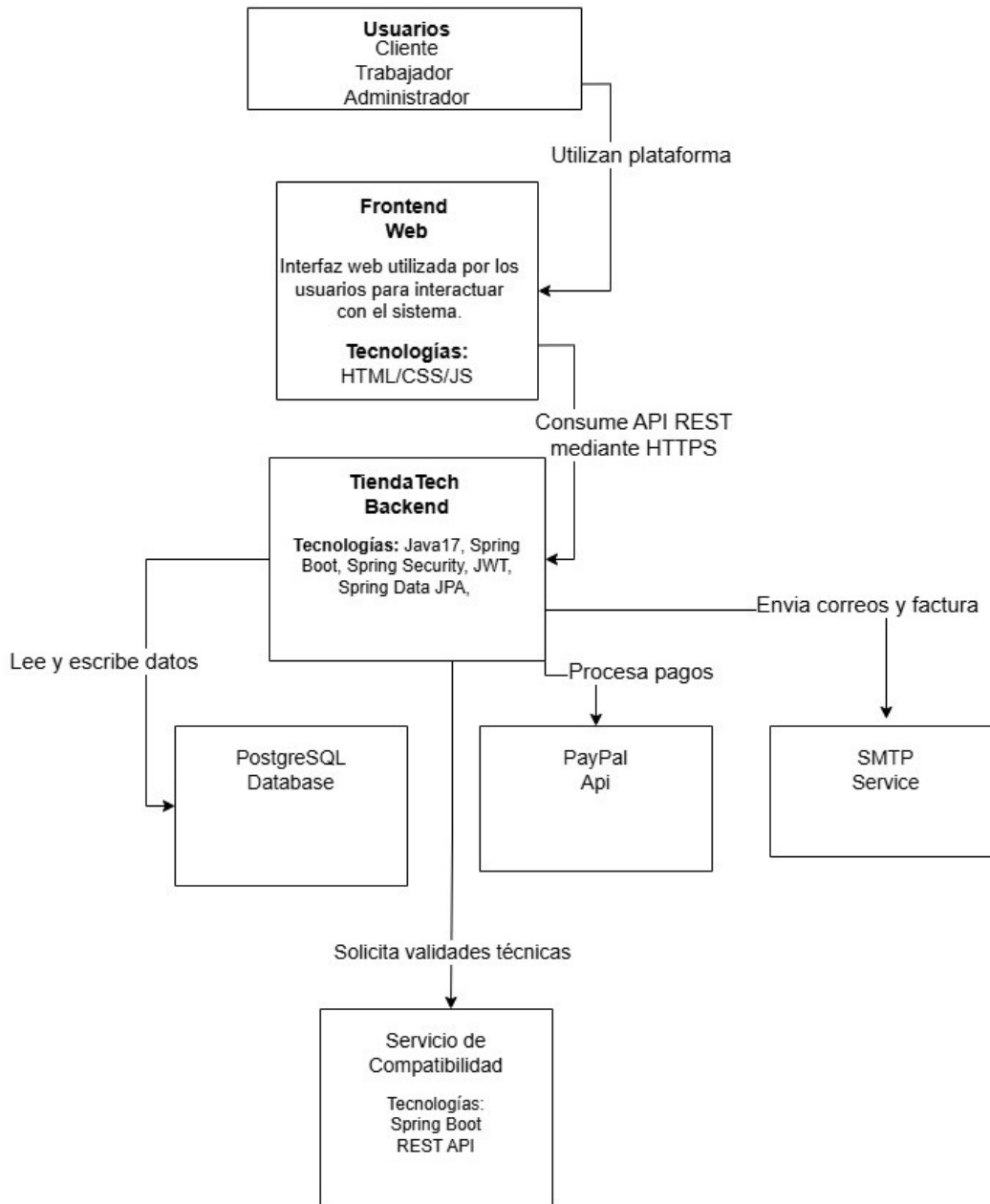


Figura 2: Diagrama C4-L2 de la arquitectura distribuida TiendaTech.

4.3. Diagrama C4-L3: Diagrama de Componentes

El diagrama C4-L3 detalla los componentes internos principales de la arquitectura de TiendaTech, tomando como base los contenedores definidos en el nivel C4-L2. En este nivel se muestra la organización interna del backend, donde se identifican módulos como autenticación, usuarios, catálogo, carrito, órdenes, pagos, facturación y notificaciones.

Además, se destaca el microservicio de compatibilidad de hardware, encargado de validar la relación técnica entre componentes como procesador, placa madre, memoria RAM, GPU, almacenamiento y fuente de poder. Esta separación permite mantener reglas técnicas independientes, mejorar la escalabilidad del sistema y evitar que errores en un módulo afecten el funcionamiento completo de la plataforma.

TiendaTech - Diagrama C4 Lvl 3: Diagrama de Componentes



Figura 3: Diagrama C4-L3 de componentes internos de TiendaTech.

4.4. Contratos API mediante OpenAPI

Los contratos API de TiendaTech se especifican mediante archivos OpenAPI 3.0.3 en formato YAML. Estos contratos definen formalmente los endpoints, métodos HTTP, parámetros, cuerpos de solicitud, respuestas y esquemas de datos utilizados por cada microservicio del sistema distribuido.

El uso de contratos API permite establecer una comunicación clara entre el frontend, el API Gateway y los microservicios internos, reduciendo ambigüedades durante la implementación. Además, facilita la documentación técnica, la validación temprana de integraciones y la generación de clientes o pruebas automatizadas a partir de una especificación común.

En TiendaTech se han definido contratos independientes para los microservicios principales: usuarios y seguridad, productos, inventario, pedidos, ventas, y órdenes con proveedores. Esta separación mantiene la responsabilidad de cada servicio delimitada y permite que cada módulo evolucione de forma independiente sin afectar directamente al resto de la arquitectura.

4.4.1. Contrato API del microservicio de usuarios y seguridad

Este contrato define las operaciones relacionadas con el registro de clientes, inicio de sesión, recuperación de contraseña, gestión de usuarios, direcciones, provincias, ciudades y roles.

```
1 # 01-usuarios-seguridad-openapi.yaml
2
3 openapi: 3.0.3
4 info:
5   title: Usuarios y Seguridad API
6   version: 1.0.0
7   description: Microservicio para registro, login, recuperación de
8     contraseña, usuarios, direcciones, provincias, ciudades y
9     roles.
10
11 servers:
12   - url: http://localhost:8081
13
14 paths:
15   /api/usuarios/registro:
16     post:
17       summary: Registrar cliente
18       tags: [Autenticación]
19       requestBody:
20         required: true
21         content:
22           application/json:
23             schema:
24               $ref: '#/components/schemas/RegistroRequest'
25       responses:
```

```

24         '201':
25             description: Cliente registrado con rol CLIENTE
26
27 /api/usuarios/login:
28     post:
29         summary: Iniciar sesi n
30         tags: [Autenticaci n]
31         requestBody:
32             required: true
33             content:
34                 application/json:
35                     schema:
36                         $ref: '#/components/schemas/LoginRequest'
37         responses:
38             '200':
39                 description: Login correcto
40                 content:
41                     application/json:
42                         schema:
43                             $ref: '#/components/schemas/LoginResponse'
44
45 /api/usuarios/recuperar-password:
46     post:
47         summary: Solicitar recuperaci n de contrase a
48         tags: [Autenticaci n]
49         requestBody:
50             required: true
51             content:
52                 application/json:
53                     schema:
54                         $ref: '#/components/schemas/RecuperarPasswordRequest'
55         responses:
56             '200':
57                 description: Solicitud de recuperaci n enviada
58
59 /api/usuarios/restablecer-password:
60     post:
61         summary: Restablecer contrase a
62         tags: [Autenticaci n]
63         requestBody:
64             required: true
65             content:
66                 application/json:
67                     schema:
68                         $ref: '#/components/schemas/RestablecerPasswordRequest'
69
70         responses:
71             '200':

```

```

71         description: Contrase a restablecida
72
73 /api/usuarios/{usuarioId}:
74   get:
75     summary: Obtener usuario por ID
76     tags: [Usuarios]
77     parameters:
78       - name: usuarioId
79         in: path
80         required: true
81         schema:
82           type: integer
83     responses:
84       '200':
85         description: Usuario encontrado
86         content:
87           application/json:
88             schema:
89               $ref: '#/components/schemas/UsuarioResponse'
90
91   put:
92     summary: Actualizar datos del usuario
93     tags: [Usuarios]
94     parameters:
95       - name: usuarioId
96         in: path
97         required: true
98         schema:
99           type: integer
100     requestBody:
101       required: true
102       content:
103         application/json:
104           schema:
105             $ref: '#/components/schemas/UsuarioUpdateRequest'
106     responses:
107       '200':
108         description: Usuario actualizado
109
110 /api/provincias:
111   get:
112     summary: Listar provincias
113     tags: [Ubicaci n]
114     responses:
115       '200':
116         description: Lista de provincias
117         content:
118           application/json:

```

```

119         schema:
120             type: array
121             items:
122                 $ref: '#/components/schemas/ProvinciaResponse'
123
124 /api/provincias/{provinciaId}/ciudades:
125     get:
126         summary: Listar ciudades por provincia
127         tags: [Ubicaci n]
128         parameters:
129             - name: provinciaId
130               in: path
131               required: true
132               schema:
133                   type: integer
134         responses:
135             '200':
136                 description: Lista de ciudades de una provincia
137                 content:
138                     application/json:
139                         schema:
140                             type: array
141                             items:
142                                 $ref: '#/components/schemas/CiudadResponse'
143
144 /api/usuarios/{usuarioId}/direcciones:
145     get:
146         summary: Listar direcciones del usuario
147         tags: [Direcciones]
148         parameters:
149             - name: usuarioId
150               in: path
151               required: true
152               schema:
153                   type: integer
154         responses:
155             '200':
156                 description: Lista de direcciones
157                 content:
158                     application/json:
159                         schema:
160                             type: array
161                             items:
162                                 $ref: '#/components/schemas/DireccionResponse'
163
164     post:
165         summary: Crear direcci n del usuario
166         tags: [Direcciones]

```

```

167     parameters:
168       - name: usuarioId
169         in: path
170         required: true
171         schema:
172           type: integer
173     requestBody:
174       required: true
175       content:
176         application/json:
177           schema:
178             $ref: '#/components/schemas/DireccionRequest'
179     responses:
180       '201':
181         description: Direcci n creada
182
183 /api/usuarios/{usuarioId}/direcciones/{direccionId}:
184 put:
185   summary: Actualizar direcci n del usuario
186   tags: [Direcciones]
187   parameters:
188     - name: usuarioId
189       in: path
190       required: true
191       schema:
192         type: integer
193     - name: direccionId
194       in: path
195       required: true
196       schema:
197         type: integer
198   requestBody:
199     required: true
200     content:
201       application/json:
202         schema:
203           $ref: '#/components/schemas/DireccionRequest'
204   responses:
205     '200':
206       description: Direcci n actualizada
207
208 delete:
209   summary: Eliminar direcci n del usuario
210   tags: [Direcciones]
211   parameters:
212     - name: usuarioId
213       in: path
214       required: true

```

```

215         schema:
216             type: integer
217     - name: direccionId
218       in: path
219       required: true
220       schema:
221           type: integer
222     responses:
223         '204':
224             description: Direcci n eliminada
225
226 /api/admin/usuarios:
227     post:
228         summary: Crear usuario trabajador
229         tags: [Administraci n]
230         requestBody:
231             required: true
232             content:
233                 application/json:
234                     schema:
235                         $ref: '#/components/schemas/TrabajadorRequest'
236         responses:
237             '201':
238                 description: Trabajador creado
239
240 /api/admin/usuarios/{usuarioId}/rol:
241     put:
242         summary: Cambiar rol de usuario
243         tags: [Administraci n]
244         parameters:
245             - name: usuarioId
246               in: path
247               required: true
248               schema:
249                   type: integer
250         requestBody:
251             required: true
252             content:
253                 application/json:
254                     schema:
255                         $ref: '#/components/schemas/RolRequest'
256         responses:
257             '200':
258                 description: Rol actualizado
259
260 /api/admin/usuarios/{usuarioId}/estado:
261     put:
262         summary: Cambiar estado del usuario

```



```

263     tags: [Administraci n]
264     parameters:
265       - name: usuarioId
266         in: path
267         required: true
268         schema:
269           type: integer
270     requestBody:
271       required: true
272       content:
273         application/json:
274           schema:
275             $ref: '#/components/schemas/EstadoUsuarioRequest'
276     responses:
277       '200':
278         description: Estado actualizado
279
280 components:
281   schemas:
282     RegistroRequest:
283       type: object
284       required: [nombres, apellidos, correo, password]
285       properties:
286         nombres:
287           type: string
288           example: Jeremy
289         apellidos:
290           type: string
291           example: Gaibor
292         correo:
293           type: string
294           format: email
295           example: cliente@gmail.com
296         password:
297           type: string
298           example: "123456"
299         telefono:
300           type: string
301           example: "0987654321"
302
303     LoginRequest:
304       type: object
305       required: [correo, password]
306       properties:
307         correo:
308           type: string
309           format: email
310           example: cliente@gmail.com

```

```

311     password:
312         type: string
313         example: "123456"
314
315 LoginResponse:
316     type: object
317     properties:
318         token:
319             type: string
320         usuarioId:
321             type: integer
322             example: 1
323         rol:
324             type: string
325             enum: [ADMIN, TRABAJADOR, CLIENTE]
326
327 RecuperarPasswordRequest:
328     type: object
329     required: [correo]
330     properties:
331         correo:
332             type: string
333             format: email
334
335 RestablecerPasswordRequest:
336     type: object
337     required: [token, nuevaPassword]
338     properties:
339         token:
340             type: string
341         nuevaPassword:
342             type: string
343
344 UsuarioUpdateRequest:
345     type: object
346     properties:
347         nombres:
348             type: string
349         apellidos:
350             type: string
351         telefono:
352             type: string
353         correo:
354             type: string
355             format: email
356
357 UsuarioResponse:
358     type: object

```

```

359     properties:
360         usuarioId:
361             type: integer
362         nombres:
363             type: string
364         apellidos:
365             type: string
366         correo:
367             type: string
368             format: email
369         telefono:
370             type: string
371         rol:
372             type: string
373             enum: [ADMIN, TRABAJADOR, CLIENTE]
374         estado:
375             type: string
376             enum: [ACTIVO, INACTIVO]
377
378 ProvinciaResponse:
379     type: object
380     properties:
381         provinciaId:
382             type: integer
383             example: 1
384         nombre:
385             type: string
386             example: Los R os
387
388 CiudadResponse:
389     type: object
390     properties:
391         ciudadId:
392             type: integer
393             example: 1
394         provinciaId:
395             type: integer
396             example: 1
397         nombre:
398             type: string
399             example: Quevedo
400
401 DireccionRequest:
402     type: object
403     required: [provinciaId, ciudadId, direccion]
404     properties:
405         provinciaId:
406             type: integer

```

```

407         example: 1
408     ciudadId:
409         type: integer
410         example: 1
411     direccion:
412         type: string
413         example: Av. Quito y calle D cima
414     referencia:
415         type: string
416         example: Frente al parque
417     telefonoContacto:
418         type: string
419         example: "0987654321"
420
421 DireccionResponse:
422     type: object
423     properties:
424         direccionId:
425             type: integer
426             example: 1
427         provinciaId:
428             type: integer
429             example: 1
430         provinciaNombre:
431             type: string
432             example: Los R os
433         ciudadId:
434             type: integer
435             example: 1
436         ciudadNombre:
437             type: string
438             example: Quevedo
439         direccion:
440             type: string
441             example: Av. Quito y calle D cima
442         referencia:
443             type: string
444             example: Frente al parque
445         telefonoContacto:
446             type: string
447             example: "0987654321"
448
449 TrabajadorRequest:
450     type: object
451     required: [nombres, apellidos, correo, password, rol]
452     properties:
453         nombres:
454             type: string

```

```

455     apellidos:
456         type: string
457     correo:
458         type: string
459         format: email
460     password:
461         type: string
462     rol:
463         type: string
464         enum: [ADMIN, TRABAJADOR]
465
466     RolRequest:
467         type: object
468         required: [rol]
469         properties:
470             rol:
471                 type: string
472                 enum: [ADMIN, TRABAJADOR, CLIENTE]
473
474     EstadoUsuarioRequest:
475         type: object
476         required: [estado]
477         properties:
478             estado:
479                 type: string
480                 enum: [ACTIVO, INACTIVO]

```

Listing 1: Contrato OpenAPI del microservicio de usuarios y seguridad

4.4.2. Contrato API del microservicio de productos

Este contrato especifica las operaciones del catálogo de productos, incluyendo la gestión de productos, categorías, marcas e imágenes asociadas.

```

1
2 openapi: 3.0.3
3 info:
4     title: Productos API
5     version: 1.0.0
6     description: Microservicio para cat logo , categor as , marcas e
7         im genes de productos.
8 servers:
9     - url: http://localhost:8081
10
11 paths:
12     /api/productos:

```

```

13  get:
14      summary: Listar productos
15      tags: [Productos]
16      responses:
17          '200':
18              description: Lista de productos
19              content:
20                  application/json:
21                      schema:
22                          type: array
23                          items:
24                              $ref: '#/components/schemas/ProductoResponse'
25
26  post:
27      summary: Crear producto
28      tags: [Productos]
29      requestBody:
30          required: true
31          content:
32              application/json:
33                  schema:
34                      $ref: '#/components/schemas/ProductoRequest'
35      responses:
36          '201':
37              description: Producto creado
38
39  /api/productos/{productId}:
40      get:
41          summary: Obtener producto por ID
42          tags: [Productos]
43          parameters:
44              - name: productId
45                in: path
46                required: true
47                schema:
48                    type: integer
49          responses:
50              '200':
51                  description: Producto encontrado
52                  content:
53                      application/json:
54                          schema:
55                              $ref: '#/components/schemas/ProductoResponse'
56
57      put:
58          summary: Actualizar producto
59          tags: [Productos]
60          parameters:

```

```

61     - name: productId
62       in: path
63       required: true
64       schema:
65         type: integer
66     requestBody:
67       required: true
68       content:
69         application/json:
70           schema:
71             $ref: '#/components/schemas/ProductoRequest'
72     responses:
73       '200':
74         description: Producto actualizado
75
76     delete:
77       summary: Eliminar producto
78       tags: [Productos]
79       parameters:
80         - name: productId
81           in: path
82           required: true
83           schema:
84             type: integer
85       responses:
86         '204':
87           description: Producto eliminado
88
89 /api/categorias:
90   get:
91     summary: Listar categor as
92     tags: [Categor as]
93     responses:
94       '200':
95         description: Lista de categor as
96         content:
97           application/json:
98             schema:
99               type: array
100             items:
101               $ref: '#/components/schemas/CategoriaResponse'
102
103   post:
104     summary: Crear categor a
105     tags: [Categor as]
106     requestBody:
107       required: true
108       content:

```

```

109         application/json:
110             schema:
111                 $ref: '#/components/schemas/CategoriaRequest'
112     responses:
113         '201':
114             description: Categoria creada
115
116 /api/marcas:
117     get:
118         summary: Listar marcas
119         tags: [Marcas]
120         responses:
121             '200':
122                 description: Lista de marcas
123                 content:
124                     application/json:
125                         schema:
126                             type: array
127                             items:
128                                 $ref: '#/components/schemas/MarcaResponse'
129
130     post:
131         summary: Crear marca
132         tags: [Marcas]
133         requestBody:
134             required: true
135             content:
136                 application/json:
137                     schema:
138                         $ref: '#/components/schemas/MarcaRequest'
139         responses:
140             '201':
141                 description: Marca creada
142
143 /api/productos/{productId}/imagen-portada:
144     post:
145         summary: Registrar imagen de portada
146         tags: [Imagenes]
147         parameters:
148             - name: productId
149               in: path
150               required: true
151               schema:
152                   type: integer
153         requestBody:
154             required: true
155             content:
156                 application/json:

```



```

157         schema:
158             $ref: '#/components/schemas/ImagenRequest'
159     responses:
160         '200':
161             description: Imagen de portada registrada
162
163 /api/productos/{productoId}/imagenes:
164     get:
165         summary: Listar im genes del producto
166         tags: [Im genes]
167         parameters:
168             - name: productoId
169               in: path
170               required: true
171               schema:
172                   type: integer
173         responses:
174             '200':
175                 description: Lista de im genes del producto
176                 content:
177                     application/json:
178                         schema:
179                             type: array
180                             items:
181                                 $ref: '#/components/schemas/ImagenResponse'
182
183     post:
184         summary: Agregar imagen a la galer a
185         tags: [Im genes]
186         parameters:
187             - name: productoId
188               in: path
189               required: true
190               schema:
191                   type: integer
192         requestBody:
193             required: true
194             content:
195                 application/json:
196                     schema:
197                         $ref: '#/components/schemas/ImagenRequest'
198         responses:
199             '201':
200                 description: Imagen agregada
201
202 components:
203     schemas:
204         ProductoRequest:

```

```

205     type: object
206     required: [nombre, precio, categoriaId, marcaId]
207     properties:
208         nombre:
209             type: string
210             example: Procesador Ryzen 5
211         descripcion:
212             type: string
213             example: Procesador AMD Ryzen 5 para computadora de
                escritorio.
214         precio:
215             type: number
216             format: double
217             example: 180.00
218         categoriaId:
219             type: integer
220             example: 1
221         marcaId:
222             type: integer
223             example: 1
224         activo:
225             type: boolean
226             example: true
227
228     ProductoResponse:
229         type: object
230         properties:
231             productoId:
232                 type: integer
233                 example: 1
234             nombre:
235                 type: string
236                 example: Procesador Ryzen 5
237             descripcion:
238                 type: string
239                 example: Procesador AMD Ryzen 5 para computadora de
                    escritorio.
240             precio:
241                 type: number
242                 format: double
243                 example: 180.00
244             categoriaId:
245                 type: integer
246                 example: 1
247             categoriaNombre:
248                 type: string
249                 example: Procesadores
250             marcaId:

```

```

251         type: integer
252         example: 1
253     marcaNombre:
254         type: string
255         example: AMD
256     imagenPortada:
257         type: string
258         example: https://servidor.com/img/ryzen5.jpg
259     activo:
260         type: boolean
261         example: true
262
263     CategoriaRequest:
264         type: object
265         required: [nombre]
266         properties:
267             nombre:
268                 type: string
269                 example: Procesadores
270             descripcion:
271                 type: string
272                 example: Componentes de procesamiento
273
274     CategoriaResponse:
275         type: object
276         properties:
277             categoriaId:
278                 type: integer
279                 example: 1
280             nombre:
281                 type: string
282                 example: Procesadores
283             descripcion:
284                 type: string
285                 example: Componentes de procesamiento
286
287     MarcaRequest:
288         type: object
289         required: [nombre]
290         properties:
291             nombre:
292                 type: string
293                 example: AMD
294             descripcion:
295                 type: string
296                 example: Marca de procesadores y tarjetas gráficas
297
298     MarcaResponse:

```

```

299     type: object
300     properties:
301       marcaId:
302         type: integer
303         example: 1
304       nombre:
305         type: string
306         example: AMD
307       descripcion:
308         type: string
309         example: Marca de procesadores y tarjetas gráficas
310
311     ImagenRequest:
312       type: object
313       required: [urlImagen]
314       properties:
315         urlImagen:
316           type: string
317           example: https://servidor.com/img/ryzen5.jpg
318         descripcion:
319           type: string
320           example: Imagen frontal del producto
321
322     ImagenResponse:
323       type: object
324       properties:
325         imagenId:
326           type: integer
327           example: 1
328         productoId:
329           type: integer
330           example: 1
331         urlImagen:
332           type: string
333           example: https://servidor.com/img/ryzen5.jpg
334         descripcion:
335           type: string
336           example: Imagen frontal del producto

```

Listing 2: Contrato OpenAPI del microservicio de productos

4.4.3. Contrato API del microservicio de inventario

Este contrato define los endpoints para consultar el inventario de productos, controlar el stock mínimo y registrar movimientos de entrada o salida de inventario.

```

1 ## 03-inventario-openapi.yaml

```

```

2 openapi: 3.0.3
3 info:
4   title: Inventario API
5   version: 1.0.0
6   description: Microservicio para stock, stock m nimo y movimientos
   de inventario.
7
8 servers:
9   - url: http://localhost:8081
10
11 paths:
12   /api/inventario/producto/{productoId}:
13     get:
14       summary: Consultar inventario de un producto
15       tags: [Inventario]
16       parameters:
17         - name: productoId
18           in: path
19           required: true
20           schema:
21             type: integer
22       responses:
23         '200':
24           description: Inventario del producto
25           content:
26             application/json:
27               schema:
28                 $ref: '#/components/schemas/InventarioResponse'
29
30   /api/inventario/stock-bajo:
31     get:
32       summary: Consultar productos con stock bajo
33       tags: [Inventario]
34       responses:
35         '200':
36           description: Lista de productos con stock bajo
37           content:
38             application/json:
39               schema:
40                 type: array
41                 items:
42                   $ref: '#/components/schemas/InventarioResponse'
43
44   /api/inventario/producto/{productoId}/stock-minimo:
45     put:
46       summary: Actualizar stock m nimo
47       tags: [Inventario]
48       parameters:

```

```

49     - name: productId
50       in: path
51       required: true
52       schema:
53         type: integer
54     requestBody:
55       required: true
56       content:
57         application/json:
58           schema:
59             $ref: '#/components/schemas/StockMinimoRequest'
60     responses:
61       '200':
62         description: Stock mínimo actualizado
63
64 /api/inventario/movimientos:
65   post:
66     summary: Registrar movimiento de inventario
67     tags: [Movimientos]
68     requestBody:
69       required: true
70       content:
71         application/json:
72           schema:
73             $ref: '#/components/schemas/MovimientoInventarioRequest'
74     responses:
75       '201':
76         description: Movimiento registrado
77
78 /api/inventario/movimientos/producto/{productId}:
79   get:
80     summary: Consultar movimientos por producto
81     tags: [Movimientos]
82     parameters:
83       - name: productId
84         in: path
85         required: true
86         schema:
87           type: integer
88     responses:
89       '200':
90         description: Movimientos del producto
91         content:
92           application/json:
93             schema:
94               type: array
95               items:

```

```

96         $ref: '#/components/schemas/
97             MovimientoInventarioResponse'
98 components:
99     schemas:
100         InventarioResponse:
101             type: object
102             properties:
103                 inventarioId:
104                     type: integer
105                     example: 1
106                 productoId:
107                     type: integer
108                     example: 1
109                 stockActual:
110                     type: integer
111                     example: 20
112                 stockMinimo:
113                     type: integer
114                     example: 5
115                 estadoStock:
116                     type: string
117                     enum: [NORMAL, BAJO, AGOTADO]
118
119         StockMinimoRequest:
120             type: object
121             required: [stockMinimo]
122             properties:
123                 stockMinimo:
124                     type: integer
125                     example: 5
126
127         MovimientoInventarioRequest:
128             type: object
129             required: [productoId, tipoMovimiento, cantidad, motivo]
130             properties:
131                 productoId:
132                     type: integer
133                     example: 1
134                 tipoMovimiento:
135                     type: string
136                     enum: [ENTRADA, SALIDA, AJUSTE]
137                 cantidad:
138                     type: integer
139                     example: 10
140                 motivo:
141                     type: string
142                     example: Recepci n de orden de compra

```

```

143
144 MovimientoInventarioResponse:
145   type: object
146   properties:
147     movimientoId:
148       type: integer
149       example: 1
150     productoId:
151       type: integer
152       example: 1
153     tipoMovimiento:
154       type: string
155       enum: [ENTRADA, SALIDA, AJUSTE]
156     cantidad:
157       type: integer
158       example: 10
159     motivo:
160       type: string
161       example: Recepci n de orden de compra
162     fecha:
163       type: string
164       format: date-time
165     stockResultante:
166       type: integer
167       example: 30

```

Listing 3: Contrato OpenAPI del microservicio de inventario

4.4.4. Contrato API del microservicio de pedidos

Este contrato contiene las operaciones relacionadas con el carrito de compras, el armado personalizado de computadoras y la gestión de pedidos generados por los usuarios.

```

1
2 ## 04-pedidos-openapi.yaml
3 openapi: 3.0.3
4 info:
5   title: Pedidos API
6   version: 1.0.0
7   description: Microservicio para carrito, armado de PC y pedidos.
8
9 servers:
10   - url: http://localhost:8081
11
12 paths:
13   /api/carrito/usuario/{usuarioId}:
14     get:

```



```

15     summary: Obtener carrito del usuario
16     tags: [Carrito]
17     parameters:
18         - name: usuarioId
19           in: path
20           required: true
21           schema:
22             type: integer
23     responses:
24         '200':
25             description: Carrito del usuario
26             content:
27                 application/json:
28                     schema:
29                         $ref: '#/components/schemas/CarritoResponse'
30
31 /api/carrito/agregar:
32     post:
33         summary: Agregar producto al carrito
34         tags: [Carrito]
35         requestBody:
36             required: true
37             content:
38                 application/json:
39                     schema:
40                         $ref: '#/components/schemas/CarritoItemRequest'
41         responses:
42             '201':
43                 description: Producto agregado al carrito
44
45 /api/carrito/item/{itemId}:
46     put:
47         summary: Actualizar cantidad del item
48         tags: [Carrito]
49         parameters:
50             - name: itemId
51               in: path
52               required: true
53               schema:
54                 type: integer
55         requestBody:
56             required: true
57             content:
58                 application/json:
59                     schema:
60                         $ref: '#/components/schemas/ActualizarCantidadRequest'
61         responses:
62             '200':

```

```

63         description: Item actualizado
64
65     delete:
66         summary: Eliminar item del carrito
67         tags: [Carrito]
68         parameters:
69             - name: itemId
70               in: path
71               required: true
72               schema:
73                 type: integer
74         responses:
75             '204':
76                 description: Item eliminado
77
78 /api/carrito/usuario/{usuarioId}/vaciar:
79     delete:
80         summary: Vaciar carrito
81         tags: [Carrito]
82         parameters:
83             - name: usuarioId
84               in: path
85               required: true
86               schema:
87                 type: integer
88         responses:
89             '204':
90                 description: Carrito vaciado
91
92 /api/armados:
93     post:
94         summary: Crear armado de PC
95         tags: [Armados]
96         requestBody:
97             required: true
98             content:
99                 application/json:
100                     schema:
101                         $ref: '#/components/schemas/ArmadoRequest'
102         responses:
103             '201':
104                 description: Armado creado
105
106 /api/armados/usuario/{usuarioId}:
107     get:
108         summary: Listar armados por usuario
109         tags: [Armados]
110         parameters:

```

```

111     - name: usuarioId
112       in: path
113       required: true
114       schema:
115         type: integer
116   responses:
117     '200':
118       description: Armados del usuario
119       content:
120         application/json:
121           schema:
122             type: array
123             items:
124               $ref: '#/components/schemas/ArmadoResponse'
125
126 /api/armados/{armadoId}/validar:
127   post:
128     summary: Validar armado de PC
129     tags: [Armados]
130     parameters:
131       - name: armadoId
132         in: path
133         required: true
134         schema:
135           type: integer
136     responses:
137       '200':
138         description: Resultado de validaci n
139         content:
140           application/json:
141             schema:
142               $ref: '#/components/schemas/ValidacionArmadoResponse'
143
144 /api/pedidos:
145   post:
146     summary: Crear pedido desde carrito
147     tags: [Pedidos]
148     requestBody:
149       required: true
150       content:
151         application/json:
152           schema:
153             $ref: '#/components/schemas/PedidoRequest'
154     responses:
155       '201':
156         description: Pedido creado
157

```

```

158 /api/pedidos/{pedidoId}:
159   get:
160     summary: Obtener pedido por ID
161     tags: [Pedidos]
162     parameters:
163       - name: pedidoId
164         in: path
165         required: true
166         schema:
167           type: integer
168     responses:
169       '200':
170         description: Pedido encontrado
171         content:
172           application/json:
173             schema:
174               $ref: '#/components/schemas/PedidoResponse'
175
176 /api/pedidos/usuario/{usuarioId}:
177   get:
178     summary: Listar pedidos por usuario
179     tags: [Pedidos]
180     parameters:
181       - name: usuarioId
182         in: path
183         required: true
184         schema:
185           type: integer
186     responses:
187       '200':
188         description: Pedidos del usuario
189         content:
190           application/json:
191             schema:
192               type: array
193               items:
194                 $ref: '#/components/schemas/PedidoResponse'
195
196 /api/pedidos/{pedidoId}/resumen:
197   get:
198     summary: Obtener resumen del pedido para ventas
199     tags: [Pedidos]
200     parameters:
201       - name: pedidoId
202         in: path
203         required: true
204         schema:
205           type: integer

```

```

206     responses:
207       '200':
208         description: Resumen del pedido
209         content:
210           application/json:
211             schema:
212               $ref: '#/components/schemas/PedidoResponse'
213
214 /api/pedidos/{pedidoId}/estado:
215   put:
216     summary: Actualizar estado del pedido
217     tags: [Pedidos]
218     parameters:
219       - name: pedidoId
220         in: path
221         required: true
222         schema:
223           type: integer
224     requestBody:
225       required: true
226       content:
227         application/json:
228           schema:
229             $ref: '#/components/schemas/EstadoPedidoRequest'
230     responses:
231       '200':
232         description: Estado actualizado
233
234 components:
235   schemas:
236     CarritoItemRequest:
237       type: object
238       required: [usuarioId, productoId, cantidad]
239       properties:
240         usuarioId:
241           type: integer
242           example: 1
243         productoId:
244           type: integer
245           example: 1
246         cantidad:
247           type: integer
248           example: 2
249
250     ActualizarCantidadRequest:
251       type: object
252       required: [cantidad]
253       properties:

```

```

254     cantidad:
255         type: integer
256         example: 3
257
258 CarritoResponse:
259     type: object
260     properties:
261         usuarioId:
262             type: integer
263             example: 1
264         items:
265             type: array
266             items:
267                 $ref: '#/components/schemas/CarritoItemResponse'
268         subtotal:
269             type: number
270             format: double
271             example: 360.00
272         impuesto:
273             type: number
274             format: double
275             example: 43.20
276         total:
277             type: number
278             format: double
279             example: 403.20
280
281 CarritoItemResponse:
282     type: object
283     properties:
284         itemId:
285             type: integer
286             example: 1
287         productId:
288             type: integer
289             example: 1
290         nombreProducto:
291             type: string
292             example: Procesador Ryzen 5
293         marcaNombre:
294             type: string
295             example: AMD
296         cantidad:
297             type: integer
298             example: 2
299         precioUnitario:
300             type: number
301             format: double

```

```

302         example: 180.00
303     subtotal:
304         type: number
305         format: double
306         example: 360.00
307
308 ArmadoRequest:
309     type: object
310     required: [usuarioId, componentes]
311     properties:
312         usuarioId:
313             type: integer
314             example: 1
315         componentes:
316             type: array
317             items:
318                 $ref: '#/components/schemas/ComponenteArmado'
319
320 ComponenteArmado:
321     type: object
322     required: [tipo, productoId]
323     properties:
324         tipo:
325             type: string
326             example: PROCESADOR
327         productoId:
328             type: integer
329             example: 1
330
331 ArmadoResponse:
332     type: object
333     properties:
334         armadoId:
335             type: integer
336             example: 1
337         usuarioId:
338             type: integer
339             example: 1
340         componentes:
341             type: array
342             items:
343                 $ref: '#/components/schemas/ComponenteArmado'
344         valido:
345             type: boolean
346             example: true
347
348 ValidacionArmadoResponse:
349     type: object

```

```

350     properties:
351         valido:
352             type: boolean
353             example: true
354         mensaje:
355             type: string
356             example: Armado compatible
357
358 PedidoRequest:
359     type: object
360     required: [usuarioId]
361     properties:
362         usuarioId:
363             type: integer
364             example: 1
365
366 PedidoResponse:
367     type: object
368     properties:
369         pedidoId:
370             type: integer
371             example: 1
372         usuarioId:
373             type: integer
374             example: 1
375         subtotal:
376             type: number
377             format: double
378             example: 360.00
379         impuesto:
380             type: number
381             format: double
382             example: 43.20
383         total:
384             type: number
385             format: double
386             example: 403.20
387         estado:
388             type: string
389             enum: [PENDIENTE, PAGADO, CANCELADO]
390         items:
391             type: array
392             items:
393                 $ref: '#/components/schemas/CarritoItemResponse'
394
395 EstadoPedidoRequest:
396     type: object
397     required: [estado]

```



```

398     properties:
399         estado:
400             type: string
401             enum: [PENDIENTE, PAGADO, CANCELADO]

```

Listing 4: Contrato OpenAPI del microservicio de pedidos

4.4.5. Contrato API del microservicio de ventas

Este contrato define las operaciones de creación de ventas, procesamiento de pagos, generación de facturas y registro de salidas de inventario.

```

1
2 ## 05-ventas-openapi.yaml
3 openapi: 3.0.3
4 info:
5     title: Ventas API
6     version: 1.0.0
7     description: Microservicio para venta final, pago, factura y
8                 salida de inventario.
9
10 servers:
11     - url: http://localhost:8081
12
13 paths:
14     /api/ventas:
15         post:
16             summary: Crear venta desde un pedido
17             tags: [Ventas]
18             requestBody:
19                 required: true
20                 content:
21                     application/json:
22                         schema:
23                             $ref: '#/components/schemas/VentaRequest'
24             responses:
25                 '201':
26                     description: Venta creada
27
28     /api/ventas/{ventaId}:
29         get:
30             summary: Obtener venta por ID
31             tags: [Ventas]
32             parameters:
33                 - name: ventaId
34                   in: path
35                   required: true

```

```

35         schema:
36             type: integer
37     responses:
38         '200':
39             description: Venta encontrada
40             content:
41                 application/json:
42                     schema:
43                         $ref: '#/components/schemas/VentaResponse'
44
45 /api/ventas/usuario/{usuarioId}:
46 get:
47     summary: Listar ventas por usuario
48     tags: [Ventas]
49     parameters:
50         - name: usuarioId
51           in: path
52           required: true
53           schema:
54               type: integer
55     responses:
56         '200':
57             description: Ventas del usuario
58             content:
59                 application/json:
60                     schema:
61                         type: array
62                         items:
63                             $ref: '#/components/schemas/VentaResponse'
64
65 /api/ventas/{ventaId}/pagar:
66 post:
67     summary: Pagar venta
68     tags: [Pagos]
69     parameters:
70         - name: ventaId
71           in: path
72           required: true
73           schema:
74               type: integer
75     requestBody:
76         required: true
77         content:
78             application/json:
79                 schema:
80                     $ref: '#/components/schemas/PagoRequest'
81     responses:
82         '200':

```

```

83         description: Pago procesado
84         content:
85             application/json:
86                 schema:
87                     $ref: '#/components/schemas/PagoResponse'
88
89 /api/ventas/{ventaId}/estado:
90     put:
91         summary: Actualizar estado de venta
92         tags: [Ventas]
93         parameters:
94             - name: ventaId
95               in: path
96               required: true
97               schema:
98                   type: integer
99         requestBody:
100             required: true
101             content:
102                 application/json:
103                     schema:
104                         $ref: '#/components/schemas/EstadoVentaRequest'
105         responses:
106             '200':
107                 description: Estado actualizado
108
109 /api/facturas/generar/{ventaId}:
110     post:
111         summary: Generar factura
112         tags: [Facturas]
113         parameters:
114             - name: ventaId
115               in: path
116               required: true
117               schema:
118                   type: integer
119         responses:
120             '201':
121                 description: Factura generada
122                 content:
123                     application/json:
124                         schema:
125                             $ref: '#/components/schemas/FacturaResponse'
126
127 /api/facturas/venta/{ventaId}:
128     get:
129         summary: Obtener factura por venta
130         tags: [Facturas]

```

```

131     parameters:
132       - name: ventaId
133         in: path
134         required: true
135         schema:
136           type: integer
137     responses:
138       '200':
139         description: Factura encontrada
140         content:
141           application/json:
142             schema:
143               $ref: '#/components/schemas/FacturaResponse'
144
145 components:
146   schemas:
147     VentaRequest:
148       type: object
149       required: [pedidoId, usuarioId, direccionId]
150       properties:
151         pedidoId:
152           type: integer
153           example: 1
154         usuarioId:
155           type: integer
156           example: 1
157         direccionId:
158           type: integer
159           example: 1
160
161     VentaResponse:
162       type: object
163       properties:
164         ventaId:
165           type: integer
166           example: 1
167         pedidoId:
168           type: integer
169           example: 1
170         usuarioId:
171           type: integer
172           example: 1
173         subtotal:
174           type: number
175           format: double
176           example: 360.00
177         impuesto:
178           type: number

```

```

179         format: double
180         example: 43.20
181     total:
182         type: number
183         format: double
184         example: 403.20
185     estado:
186         type: string
187         enum: [PENDIENTE_PAGO, PAGADA, CANCELADA]
188     direccionEnvio:
189         type: string
190         example: Los R os , Quevedo, Av. Quito y calle D cima
191     fecha:
192         type: string
193         format: date-time
194
195 PagoRequest:
196     type: object
197     required: [metodoPago]
198     properties:
199         metodoPago:
200             type: string
201             enum: [TARJETA, PAYPAL]
202         numeroTarjeta:
203             type: string
204             example: "4111111111111111"
205         fechaExpiracion:
206             type: string
207             example: "12/28"
208         cvv:
209             type: string
210             example: "123"
211         correoPaypal:
212             type: string
213             example: cliente@paypal.com
214
215 PagoResponse:
216     type: object
217     properties:
218         pagoId:
219             type: integer
220             example: 1
221         ventaId:
222             type: integer
223             example: 1
224         metodoPago:
225             type: string
226             enum: [TARJETA, PAYPAL]

```

```

227     ultimos4Tarjeta:
228         type: string
229         example: "1111"
230     estadoPago:
231         type: string
232         enum: [APROBADO, RECHAZADO, PENDIENTE]
233     mensaje:
234         type: string
235         example: Pago aprobado correctamente
236
237 EstadoVentaRequest:
238     type: object
239     required: [estado]
240     properties:
241         estado:
242             type: string
243             enum: [PENDIENTE_PAGO, PAGADA, CANCELADA]
244
245 FacturaResponse:
246     type: object
247     properties:
248         facturaId:
249             type: integer
250             example: 1
251         ventaId:
252             type: integer
253             example: 1
254         numeroFactura:
255             type: string
256             example: F001-0000001
257         subtotal:
258             type: number
259             format: double
260             example: 360.00
261         impuesto:
262             type: number
263             format: double
264             example: 43.20
265         total:
266             type: number
267             format: double
268             example: 403.20
269         fechaEmision:
270             type: string
271             format: date-time

```

Listing 5: Contrato OpenAPI del microservicio de ventas

4.4.6. Contrato API del microservicio de órdenes y proveedores

Este contrato especifica la gestión de proveedores, productos suministrados y órdenes de compra, permitiendo controlar el abastecimiento de productos dentro del ecosistema TiendaTech.

```
1 ## 06-ordenes-proveedores-openapi.yaml
2 openapi: 3.0.3
3 info:
4   title:   rdenes   y Proveedores API
5   version: 1.0.0
6   description: Microservicio para proveedores, productos
7     suministrados y rdenes de compra.
8
9 servers:
10  - url: http://localhost:8081
11
12 paths:
13   /api/proveedores:
14     get:
15       summary: Listar proveedores
16       tags: [Proveedores]
17       responses:
18         '200':
19           description: Lista de proveedores
20           content:
21             application/json:
22               schema:
23                 type: array
24                 items:
25                   $ref: '#/components/schemas/ProveedorResponse'
26
27     post:
28       summary: Crear proveedor
29       tags: [Proveedores]
30       requestBody:
31         required: true
32         content:
33           application/json:
34             schema:
35               $ref: '#/components/schemas/ProveedorRequest'
36       responses:
37         '201':
38           description: Proveedor creado
39
40   /api/proveedores/{proveedorId}:
41     get:
42       summary: Obtener proveedor por ID
```

```

43     tags: [Proveedores]
44     parameters:
45       - name: proveedorId
46         in: path
47         required: true
48         schema:
49           type: integer
50     responses:
51       '200':
52         description: Proveedor encontrado
53         content:
54           application/json:
55             schema:
56               $ref: '#/components/schemas/ProveedorResponse'
57
58   put:
59     summary: Actualizar proveedor
60     tags: [Proveedores]
61     parameters:
62       - name: proveedorId
63         in: path
64         required: true
65         schema:
66           type: integer
67     requestBody:
68       required: true
69       content:
70         application/json:
71           schema:
72             $ref: '#/components/schemas/ProveedorRequest'
73     responses:
74       '200':
75         description: Proveedor actualizado
76
77   delete:
78     summary: Eliminar proveedor
79     tags: [Proveedores]
80     parameters:
81       - name: proveedorId
82         in: path
83         required: true
84         schema:
85           type: integer
86     responses:
87       '204':
88         description: Proveedor eliminado
89
90 /api/proveedores/{proveedorId}/productos:

```



```

91  get:
92      summary: Listar productos suministrados por proveedor
93      tags: [Proveedor Productos]
94      parameters:
95          - name: proveedorId
96            in: path
97            required: true
98            schema:
99                type: integer
100     responses:
101         '200':
102             description: Productos del proveedor
103             content:
104                 application/json:
105                     schema:
106                         type: array
107                         items:
108                             $ref: '#/components/schemas/
109                                 ProveedorProductoResponse'
110
111     post:
112         summary: Asociar producto a proveedor
113         tags: [Proveedor Productos]
114         parameters:
115             - name: proveedorId
116               in: path
117               required: true
118               schema:
119                   type: integer
120         requestBody:
121             required: true
122             content:
123                 application/json:
124                     schema:
125                         $ref: '#/components/schemas/ProveedorProductoRequest'
126         responses:
127             '201':
128                 description: Producto asociado al proveedor
129
130 /api/ordenes-compra:
131     get:
132         summary: Listar rdenes de compra
133         tags: [ rdenes de Compra]
134         responses:
135             '200':
136                 description: Lista de rdenes de compra
137                 content:
138                     application/json:

```

```

138         schema:
139             type: array
140             items:
141                 $ref: '#/components/schemas/OrdenCompraResponse'
142
143     post:
144         summary: Crear orden de compra
145         tags: [ rdenes de Compra]
146         requestBody:
147             required: true
148             content:
149                 application/json:
150                     schema:
151                         $ref: '#/components/schemas/OrdenCompraRequest'
152         responses:
153             '201':
154                 description: Orden creada
155
156 /api/ordenes-compra/{ordenId}:
157     get:
158         summary: Obtener orden de compra por ID
159         tags: [ rdenes de Compra]
160         parameters:
161             - name: ordenId
162               in: path
163               required: true
164               schema:
165                   type: integer
166         responses:
167             '200':
168                 description: Orden encontrada
169                 content:
170                     application/json:
171                         schema:
172                             $ref: '#/components/schemas/OrdenCompraResponse'
173
174 /api/ordenes-compra/proveedor/{proveedorId}:
175     get:
176         summary: Listar rdenes por proveedor
177         tags: [ rdenes de Compra]
178         parameters:
179             - name: proveedorId
180               in: path
181               required: true
182               schema:
183                   type: integer
184         responses:
185             '200':

```

```

186         description: rdenes del proveedor
187         content:
188             application/json:
189                 schema:
190                     type: array
191                     items:
192                         $ref: '#/components/schemas/OrdenCompraResponse'
193
194 /api/ordenes-compra/{ordenId}/confirmar-recepcion:
195     put:
196         summary: Confirmar recepci n de mercanc a
197         tags: [ rdenes de Compra]
198         parameters:
199             - name: ordenId
200               in: path
201               required: true
202               schema:
203                   type: integer
204         responses:
205             '200':
206                 description: Recepci n confirmada
207
208 components:
209     schemas:
210         ProveedorRequest:
211             type: object
212             required: [nombre, telefono]
213             properties:
214                 nombre:
215                     type: string
216                     example: Importadora PC
217                 telefono:
218                     type: string
219                     example: "0987654321"
220                 correo:
221                     type: string
222                     format: email
223                     example: proveedor@gmail.com
224                 direccion:
225                     type: string
226                     example: Guayaquil, centro
227                 condicionesPago:
228                     type: string
229                     example: Pago contra entrega
230                 calificacion:
231                     type: number
232                     format: double
233                     example: 4.5

```

```

234
235 ProveedorResponse:
236   type: object
237   properties:
238     proveedorId:
239       type: integer
240       example: 1
241     nombre:
242       type: string
243       example: Importadora PC
244     telefono:
245       type: string
246       example: "0987654321"
247     correo:
248       type: string
249       format: email
250       example: proveedor@gmail.com
251     direccion:
252       type: string
253       example: Guayaquil, centro
254     condicionesPago:
255       type: string
256       example: Pago contra entrega
257     calificacion:
258       type: number
259       format: double
260       example: 4.5
261     estado:
262       type: string
263       enum: [ACTIVO, INACTIVO]
264
265 ProveedorProductoRequest:
266   type: object
267   required: [productoId, precioCosto]
268   properties:
269     productoId:
270       type: integer
271       example: 1
272     precioCosto:
273       type: number
274       format: double
275       example: 120.00
276
277 ProveedorProductoResponse:
278   type: object
279   properties:
280     proveedorProductoId:
281       type: integer

```

```

282         example: 1
283     proveedorId:
284         type: integer
285         example: 1
286     productoId:
287         type: integer
288         example: 1
289     productoNombre:
290         type: string
291         example: Procesador Ryzen 5
292     marcaNombre:
293         type: string
294         example: AMD
295     precioCosto:
296         type: number
297         format: double
298         example: 120.00
299
300 OrdenCompraRequest:
301     type: object
302     required: [proveedorId, usuarioId, detalles]
303     properties:
304         proveedorId:
305             type: integer
306             example: 1
307         usuarioId:
308             type: integer
309             example: 1
310         detalles:
311             type: array
312             items:
313                 $ref: '#/components/schemas/OrdenCompraDetalleRequest'
314
315 OrdenCompraDetalleRequest:
316     type: object
317     required: [productoId, cantidad, precioCosto]
318     properties:
319         productoId:
320             type: integer
321             example: 1
322         cantidad:
323             type: integer
324             example: 10
325         precioCosto:
326             type: number
327             format: double
328             example: 120.00
329

```

```

330   OrdenCompraResponse:
331     type: object
332     properties:
333       ordenId:
334         type: integer
335         example: 1
336       proveedorId:
337         type: integer
338         example: 1
339       usuarioId:
340         type: integer
341         example: 1
342       total:
343         type: number
344         format: double
345         example: 1200.00
346       estado:
347         type: string
348         enum: [PENDIENTE, RECIBIDA, CANCELADA]
349       fecha:
350         type: string
351         format: date-time
352       detalles:
353         type: array
354         items:
355           $ref: '#/components/schemas/OrdenCompraDetalleRequest'

```

Listing 6: Contrato OpenAPI del microservicio de órdenes y proveedores

4.5. Estrategia de contenerización del sistema distribuido

4.5.0.1 Tecnología utilizada

Cuadro 2: Herramientas utilizadas en el entorno de despliegue.

Herramienta	Versión	Propósito
Docker	24+	Contenerización de cada microservicio.
Docker Compose	V2	Orquestación de todos los contenedores.
PostgreSQL	17	Motor de base de datos por contenedor.
Eclipse Temurin JRE	17	Entorno de ejecución.
Maven	3.9.9	Compilación en etapa de <i>build</i> .

4.5.0.2 Estrategia de Bases de Datos

Cada microservicio tiene su propia base de datos PostgreSQL aislada.

Cuadro 3: Asignación de contenedores de base de datos por microservicio.

Microservicio	Contenedor BD	Puerto externo
Usuarios y Seguridad	tiendatech_db_usuarios	5433
Productos	tiendatech_db_productos	5434
Inventario	tiendatech_db_inventario	5435
Pedidos	tiendatech_db_pedidos	5436
Ventas	tiendatech_db_ventas	5437
Órdenes y Proveedores	tiendatech_db_ordenes	5438

Los datos se persisten mediante volúmenes Docker, lo que asegura que no se pierdan al reiniciar los contenedores.

4.5.0.3 Comunicación entre microservicios

Todos los contenedores se conectan dentro de una red interna llamada `tiendatech_net` de tipo **bridge**. Dentro de esta red, cada servicio se referencia por el **nombre de su contenedor** en lugar de una IP:

Ejemplos de comunicación interna

- `pedidos-service` → `http://inventario-service:8083/api/inventario/...`
- `ventas-service` → `http://pedidos-service:8084/api/pedidos/...`

Esto hace que la comunicación sea independiente de la IP del host.

4.5.0.4 Variables de entorno

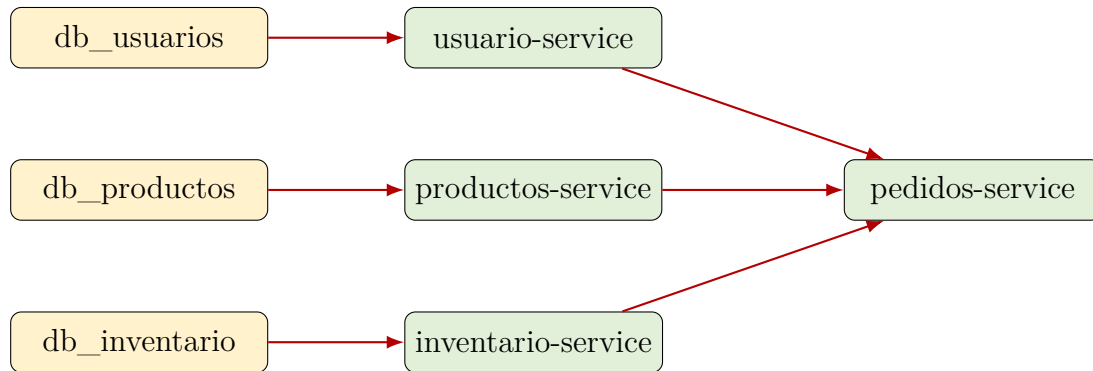
La configuración sensible, como credenciales, URLs de servicios y claves, se inyecta mediante variables de entorno en el archivo `docker-compose.yml`, evitando hardcodear datos en el código fuente.

Ejemplo para el microservicio de Ventas:

```
environment:
  SPRING_DATASOURCE_URL: jdbc:postgresql://db_ventas:5432/db_ventas
  PEDIDOS_SERVICE_URL: http://pedidos-service:8084
  PAYPAL_CLIENT_ID: ${PAYPAL_CLIENT_ID}
  PAYPAL_CLIENT_SECRET: ${PAYPAL_CLIENT_SECRET}
```

4.5.0.5 Orden de arranque – depends_on

Se define el orden de inicio mediante **depends_on** para evitar que un microservicio intente conectarse a su base de datos o a otro servicio antes de que estén disponibles:



4.5.0.6 Puertos expuestos

Cuadro 4: Puertos expuestos por los contenedores del sistema.

Contenedor	Puerto interno	Puerto externo
usuarios-service	8081	8081
productos-service	8082	8082
inventario-service	8083	8083
pedidos-service	8084	8084
ventas-service	8085	8085
ordenes-service	8086	8086
db_usuarios	5432	5433
db_productos	5432	5434
db_inventario	5432	5435
db_pedidos	5432	5436
db_ventas	5432	5437
db_ordenes	5432	5438

4.5.0.7 Comandos de despliegue

Para levantar todo el sistema:

```
docker-compose up --build
```

Para detener y eliminar contenedores:


```
docker-compose down
```

Para detener conservando los volúmenes, es decir, los datos de la base de datos:

```
docker-compose stop
```

4.5.0.8 Dockerfile

```
# syntax=docker/dockerfile:1
# -----
# Etapa 1: Build - compila el proyecto Java
# -----
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /workspace

# Forzar codificacion UTF-8 (necesario para espanol)
ENV LANG=C.UTF-8
ENV LC_ALL=C.UTF-8

# Cachea dependencias antes de copiar el codigo
COPY pom.xml .
RUN mvn -q -DskipTests dependency:go-offline

# Copia el codigo fuente y compila
COPY src ./src
RUN mvn -DskipTests clean package

# -----
# Etapa 2: Runtime - imagen ligera para correr
# -----
FROM eclipse-temurin:17-jre
WORKDIR /app

# Copia solo el JAR generado en la etapa anterior
COPY --from=build /workspace/target/*-SNAPSHOT.jar app.jar
ENV JAVA_OPTS=""

# Expone el puerto del microservicio (se sobreescribe por variable)
EXPOSE 8080
CMD ["sh", "-c", "java $JAVA_OPTS -Dserver.port=${PORT:-8080} -jar app.jar"]
```

4.5.0.9 Docker Compose

```
# =====
# TiendaTech - Docker Compose
# Levanta los 6 microservicios + 6 bases de datos PostgreSQL
```

```

# Uso: docker-compose up --build
# =====
services:
  # -----
  # MICROSERVICIO: Usuarios y Seguridad
  # Puerto: 8081 | Java / Spring Boot
  # -----
  usuarios-service:
    build:
      context: ./usuarios-service
      dockerfile: Dockerfile.java
    container_name: tiendatech_usuarios
    restart: always
    ports:
      - "8081:8081"
    environment:
      PORT: 8081
      SPRING_DATASOURCE_URL: jdbc:postgresql://db_usuarios:5432/db_usuarios
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: 12345
      JWT_SECRET: tiendatech_jwt_secret_key
    depends_on:
      - db_usuarios
    networks:
      - tiendatech_net

  db_usuarios:
    image: postgres:17
    container_name: tiendatech_db_usuarios
    restart: always
    environment:
      POSTGRES_DB: db_usuarios
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: 12345
    ports:
      - "5433:5432"
    volumes:
      - vol_db_usuarios:/var/lib/postgresql/data
    networks:
      - tiendatech_net

  # -----
  # MICROSERVICIO: Productos
  # Puerto: 8082 | Python / FastAPI
  # -----
  productos-service:
    build:
      context: ./productos-service

```

```

    dockerfile: Dockerfile.java
    container_name: tiendatech_productos
    restart: always
    ports:
      - "8082:8082"
    environment:
      PORT: 8082
      SPRING_DATASOURCE_URL: jdbc:postgresql://db_productos:5432/db_productos
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: 12345
    depends_on:
      - db_productos
    networks:
      - tiendatech_net

db_productos:
  image: postgres:17
  container_name: tiendatech_db_productos
  restart: always
  environment:
    POSTGRES_DB: db_productos
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: 12345
  ports:
    - "5434:5432"
  volumes:
    - vol_db_productos:/var/lib/postgresql/data
  networks:
    - tiendatech_net

# -----
# MICROSERVICIO: Inventario
# Puerto: 8083 | Java / Spring Boot
# -----
inventario-service:
  build:
    context: ./inventario-service
    dockerfile: Dockerfile.java
  container_name: tiendatech_inventario
  restart: always
  ports:
    - "8083:8083"
  environment:
    PORT: 8083
    SPRING_DATASOURCE_URL: jdbc:postgresql://db_inventario:5432/
    db_inventario
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: 12345

```

```

    PRODUCTOS_SERVICE_URL: http://productos-service:8082
depends_on:
  - db_inventario
  - productos-service
networks:
  - tiendatech_net

db_inventario:
  image: postgres:17
  container_name: tiendatech_db_inventario
  restart: always
  environment:
    POSTGRES_DB: db_inventario
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: 12345
  ports:
    - "5435:5432"
  volumes:
    - vol_db_inventario:/var/lib/postgresql/data
  networks:
    - tiendatech_net

# -----
# MICROSERVICIO: Pedidos
# Puerto: 8084 | Java / Spring Boot
# -----
pedidos-service:
  build:
    context: ./pedidos-service
    dockerfile: Dockerfile.java
  container_name: tiendatech_pedidos
  restart: always
  ports:
    - "8084:8084"
  environment:
    PORT: 8084
    SPRING_DATASOURCE_URL: jdbc:postgresql://db_pedidos:5432/db_pedidos
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: 12345
    USUARIOS_SERVICE_URL: http://usuarios-service:8081
    PRODUCTOS_SERVICE_URL: http://productos-service:8082
    INVENTARIO_SERVICE_URL: http://inventario-service:8083
  depends_on:
    - db_pedidos
    - usuarios-service
    - productos-service
    - inventario-service
  networks:

```

```

    - tiendatech_net

db_pedidos:
  image: postgres:17
  container_name: tiendatech_db_pedidos
  restart: always
  environment:
    POSTGRES_DB: db_pedidos
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: 12345
  ports:
    - "5436:5432"
  volumes:
    - vol_db_pedidos:/var/lib/postgresql/data
  networks:
    - tiendatech_net

# -----
# MICROSERVICIO: Ventas
# Puerto: 8085 | Java / Spring Boot
# -----
ventas-service:
  build:
    context: ./ventas-service
    dockerfile: Dockerfile.java
  container_name: tiendatech_ventas
  restart: always
  ports:
    - "8085:8085"
  environment:
    PORT: 8085
    SPRING_DATASOURCE_URL: jdbc:postgresql://db_ventas:5432/db_ventas
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: 12345
    PEDIDOS_SERVICE_URL: http://pedidos-service:8084
    USUARIOS_SERVICE_URL: http://usuarios-service:8081
    INVENTARIO_SERVICE_URL: http://inventario-service:8083
    PAYPAL_CLIENT_ID: ${PAYPAL_CLIENT_ID}
    PAYPAL_CLIENT_SECRET: ${PAYPAL_CLIENT_SECRET}
  depends_on:
    - db_ventas
    - pedidos-service
    - usuarios-service
    - inventario-service
  networks:
    - tiendatech_net

db_ventas:

```

```

image: postgres:17
container_name: tiendatech_db_ventas
restart: always
environment:
  POSTGRES_DB: db_ventas
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: 12345
ports:
  - "5437:5432"
volumes:
  - vol_db_ventas:/var/lib/postgresql/data
networks:
  - tiendatech_net

# -----
# MICROSERVICIO: Ordenes y Proveedores
# Puerto: 8086 | Java / Spring Boot
# -----
ordenes-service:
  build:
    context: ./ordenes-service
    dockerfile: Dockerfile.java
  container_name: tiendatech_ordenes
  restart: always
  ports:
    - "8086:8086"
  environment:
    PORT: 8086
    SPRING_DATASOURCE_URL: jdbc:postgresql://db_ordenes:5432/db_ordenes
    SPRING_DATASOURCE_USERNAME: postgres
    SPRING_DATASOURCE_PASSWORD: 12345
    USUARIOS_SERVICE_URL: http://usuarios-service:8081
    PRODUCTOS_SERVICE_URL: http://productos-service:8082
    INVENTARIO_SERVICE_URL: http://inventario-service:8083
  depends_on:
    - db_ordenes
    - usuarios-service
    - productos-service
    - inventario-service
  networks:
    - tiendatech_net

db_ordenes:
  image: postgres:17
  container_name: tiendatech_db_ordenes
  restart: always
  environment:
    POSTGRES_DB: db_ordenes

```

```

    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: 12345
ports:
  - "5438:5432"
volumes:
  - vol_db_ordenes:/var/lib/postgresql/data
networks:
  - tiendatech_net

# -----
# RED: todos los servicios se comunican
# por nombre de contenedor dentro de esta red
# -----
networks:
  tiendatech_net:
    driver: bridge

# -----
# VOLUMENES: persistencia de datos de cada BD
# -----
volumes:
  vol_db_usuarios:
  vol_db_productos:
  vol_db_inventario:
  vol_db_pedidos:
  vol_db_ventas:
  vol_db_ordenes:

```

4.6. Mapeo de Problemas y Enfoques Metodológicos

El primer núcleo problemático identificado en la literatura corresponde a la rigidez y la falta de contextualización en las recomendaciones tradicionales de catálogo. Trabajos como el de Vinayababu y Raju [1] y Khadka y Sthapit [6] abordan la transición desde el filtrado colaborativo clásico hacia arquitecturas híbridas distribuidas, demostrando que la diversificación y la escalabilidad del catálogo son críticas para mitigar la sobrecarga de información en entornos transaccionales de alta concurrencia. En este sentido, la optimización matemática de estos catálogos ha sido evaluada empíricamente por Tokala et al. [7], quien demuestra que variantes avanzadas de factorización de matrices (como SVD++) minimizan el error cuadrático medio (RMSE) en el backend analítico de las plataformas comerciales.

El segundo desafío crítico radica en la resolución de colisiones lógicas durante la personalización adaptativa de productos. El enfoque metodológico de Velten y Hamkins [3] introduce la programación por restricciones (Constraint Programming) como una solución óptima para modelar tablas de restricciones de ingeniería informando en tiempo real al usuario sobre conflictos mínimos y sus respectivas resoluciones. Para complementar este dinamismo sin de-

gradar la experiencia de usuario, Ben Ncir y Alattas [4] proponen modelos de *co-clustering* explicable que no solo agrupan las preferencias multidimensionales de los clientes, sino que dotan al sistema de interpretabilidad, explicando de forma transparente la razón técnica de una sugerencia o restricción.

Finalmente, la brecha de interacción interactiva se ha visto revolucionada por el procesamiento de lenguaje natural. Chihab et al. [8] demuestran la viabilidad de clasificar automáticamente perfiles de usuarios a partir de texto no estructurado para dirigir recomendaciones basadas en competencias o necesidades explícitas. Asimismo, Munson et al. [5] y Sabiri et al. [2] exponen la cúspide actual de esta evolución: la hibridación de Modelos de Lenguaje Grandes (LLMs) con sistemas de recomendación tradicionales, transitando de estructuras rígidas basadas en matrices hacia sistemas cognitivos capaces de mantener diálogos fluidos utilizando arquitecturas Transformer y recuperación aumentada de datos (RAG).

4.7. Tabla Comparativa de Trabajos Relacionados

Para sintetizar los enfoques metodológicos y contrastar el estado actual de la tecnología, la Tabla 5 condensa las contribuciones, ventajas y limitaciones de las fuentes indexadas seleccionadas.

Cuadro 5: Matriz comparativa del Estado del Arte.

Referencia Indexada	Enfoque Metodológico / Tecnología	Ventajas Principales	Limitaciones Detectadas
Vinayababu y Raju [1]	Encuesta de Sistemas Híbridos	Mapeo global de arquitecturas modernas.	No profundiza en validaciones lógicas de ingeniería.
Sabiri et al. [2]	Sistemas basados en Calidad y Microservicios	Alta disponibilidad y métricas de rendimiento.	Complejidad elevada en el acoplamiento de datos.
Chihab et al. [8]	NLP y Clasificación Supervisada	Clasificación precisa de perfiles (96.38 %).	Requiere datos pre-etiquetados masivos.
Ben Ncir y Alattas [4]	Overlapping Co-Clustering (OCCMM)	Alta interpretabilidad y explicabilidad del sistema.	Elevado consumo computacional en matrices densas.
Tokala et al. [7]	Factorización de Matrices (SVD/SVD++)	Optimización matemática predictiva del catálogo.	Es un proceso estático, no apto para flujos síncronos.
Velten y Hamkins [3]	Programación por Restricciones (CP-Solver)	Resolución de conflictos mínimos en tiempo real.	Limitado a configuraciones industriales rígidas.
Munson et al. [5]	Modelos de Lenguaje Grande (LLMs)	Interacción conversacional altamente contextual.	Riesgo latente de alucinación de datos técnicos.

4.8. Declaración del Vacío Científico (Gap Statement)

A pesar de los significativos avances evidenciados en el mapeo previo, la revisión de la literatura científica indexada expone un vacío técnico crítico (*Gap Statement*): ****existe una desconexión metodológica entre los entornos de asistencia conversacional inteligente basados en LLMs y los motores rígidos de validación de ingeniería por restricciones lógicas operando en arquitecturas distribuidas de alta disponibilidad.**** Mientras que las investigaciones actuales solucionan el procesamiento conversacional de forma aislada [5], o resuelven las colisiones de componentes de manera puramente matemática y confinada a sistemas industriales cerrados [3], no se evidencia un marco arquitectónico integrado en el e-commerce de consumo masivo que unifique ambos mundos bajo microservicios elásticos. La mayoría de los desarrollos conversacionales carecen de un motor de restricciones síncrono que impida la alucinación de compatibilidades de hardware informático, mientras que los solvers de restricciones tradicionales carecen de interfaces cognitivas en lenguaje natural accesibles para el usuario común. El proyecto **TiendaTech** se propone mitigar este vacío científico, contribuyendo con el diseño y

despliegue de un ecosistema distribuido donde un API Gateway orquesta concurrentemente un asistente conversacional RAG y un motor de restricciones elástico de compatibilidad de hardware a escala nacional.

4.9. Registros de decisiones arquitectónicas ADR

4.9.1. ADR-01: Arquitectura basada en microservicios

Estado: Aceptado.

Alternativas consideradas: Arquitectura monolítica, descartada por el alto acoplamiento entre módulos; y arquitectura en capas N-tier, descartada por sus limitaciones para escalar servicios de forma independiente.

Decisión: Se adopta una arquitectura basada en microservicios, donde los módulos principales del sistema funcionan como servicios independientes: autenticación, catálogo, carrito, pagos, facturación, motor de compatibilidad y asistente inteligente.

Justificación: Esta arquitectura permite separar responsabilidades, escalar módulos específicos y reducir el impacto de fallos. Por ejemplo, un problema en el catálogo no debería afectar directamente el procesamiento de pagos.

Consecuencia: Mejora la escalabilidad, disponibilidad y mantenibilidad del sistema, aunque incrementa la complejidad de comunicación entre servicios.

4.9.2. ADR-02: Uso de API Gateway

Estado: Aceptado.

Alternativas consideradas: Comunicación directa entre cliente y microservicios, descartada por riesgos de seguridad y mayor acoplamiento; y Backend for Frontend, descartado porque en esta fase solo se contempla un cliente web principal.

Decisión: Se implementa un API Gateway como punto de entrada único para las solicitudes del cliente web.

Justificación: El API Gateway centraliza el enrutamiento, la autenticación y el control de acceso hacia los microservicios. Esto evita que el cliente se comunique directamente con cada servicio y mejora la seguridad del sistema.

Consecuencia: Facilita el control de solicitudes y la organización de la arquitectura, pero se convierte en un componente crítico que debe manejarse con tolerancia a fallos.

4.9.3. ADR-03: Motor de compatibilidad como microservicio independiente

Estado: Aceptado.

Alternativas consideradas: Incluir la lógica de compatibilidad dentro del catálogo, descartado por mezclar responsabilidades; y validar la compatibilidad en el frontend, descartado por riesgos de manipulación y baja confiabilidad.

Decisión: Se implementa el motor de compatibilidad de hardware como un microservicio especializado.

Justificación: La validación entre procesador, placa madre, RAM, GPU, almacenamiento y fuente de poder requiere reglas técnicas que pueden cambiar con frecuencia. Separar este módulo permite actualizar las reglas sin modificar el catálogo, el carrito o los pagos.

Consecuencia: Mejora la precisión y mantenibilidad del sistema, aunque requiere comunicación constante con el catálogo de productos para mantener actualizada la información técnica.

4.10. Métricas de calidad basadas en ISO/IEC 25010

Las métricas se definen para evaluar atributos de calidad relevantes en **TiendaTech**, especialmente rendimiento, disponibilidad, fiabilidad, seguridad, mantenibilidad y compatibilidad funcional.

Cuadro 6: Métricas de calidad basadas en ISO/IEC 25010 para TiendaTech.

Característica	Métrica conocida	Forma de medición / fórmula	Umbral objetivo
Rendimiento	Tiempo de respuesta P95	Ordenar los tiempos de respuesta y tomar el valor donde se ubica el 95 % de solicitudes.	≤ 500 ms
Rendimiento	Throughput	$\frac{\text{Total de solicitudes procesadas}}{\text{Tiempo total de prueba}}$	≥ 30 req/s
Disponibilidad	Uptime	$\frac{\text{Tiempo activo del sistema}}{\text{Tiempo total de operación}} \times 100$	$\geq 99\%$
Fiabilidad	Error Rate	$\frac{\text{Solicitudes fallidas}}{\text{Total de solicitudes}} \times 100$	$\leq 1\%$
Mantenibilidad	MTTR	$\frac{\text{Tiempo total de reparación}}{\text{Número de incidentes corregidos}}$	≤ 2 horas
Seguridad	Tasa de bloqueo de accesos no autorizados	$\frac{\text{Intentos no autorizados bloqueados}}{\text{Total de intentos no autorizados}} \times 100$	$\geq 99\%$
Compatibilidad funcional	Tasa de pruebas funcionales aprobadas	$\frac{\text{Casos de prueba aprobados}}{\text{Total de casos de prueba ejecutados}} \times 100$	$\geq 95\%$

Estas métricas cubren los aspectos más importantes de **TiendaTech**. En esta fase se priorizan el rendimiento, la disponibilidad, la fiabilidad, la seguridad y la compatibilidad funcional, porque afectan directamente la experiencia del usuario y la correcta validación de los componentes de hardware.